InstrumentProject

Class Design Document

Version: 0.1 (Draft version)

Authors:

No.	Student ID	Student name
<số máy>	<mã sinh viên>	<họ tên sinh viên>

The purpose of this document is the draft version of class design that provides a guideline for detailing class design and writing the class diagram description for the solution based on design pattern approach.

Points to remember

- Content is important, not the volume. Another team should be able to design in more detail from this document.
- Pay attention to overall consistency between problem, solution, class diagram, and code example.
- Sinh viên được phép xem tài liệu nhưng không được sao chép hay phổ biến tài liệu trong phòng thi dưới mọi hình thức.

Contents

1. Name 3
2. Problem 3
3. Solution 3
3.1. Class diagram 4
3.2. Class Descriptions 5
3.2.1. Class: Instrument 5
3.2.2. Class: Handy 7
3.2.3. Class: Station 7
4. Consequence 8
5. Implementation (Java code example) 8

1. Name:

InstrumentProject - Class Design using Strategy Pattern and Template Method Pattern for IoT Instrumentation

2. Problem

IoTLab cần thiết kế phần mềm nhúng cho các dụng cụ đo môi trường. Mỗi dụng cụ có thể gắn các cảm biến thuộc 4 loại DO, PH, SALINITY và NH4; đồng thời sử dụng một trong 3 thiết bị truyền tin là WIFI, ZigBee hoặc LoRa. Có 2 loại dụng cụ: Handy chỉ gắn đúng 1 cảm biến, còn Station gắn từ 1 đến 4 cảm biến. Hệ thống phải cho phép đọc thời gian thực, đọc dữ liệu cảm biến theo định dạng JSON, lưu dữ liệu, rồi truyền dữ liệu về Data Server. Ngoài ra thiết kế phải dễ mở rộng khi thêm loại cảm biến hoặc loại thiết bị truyền tin mới.

3. Solution

Giải pháp đề xuất kết hợp Strategy pattern và Template Method pattern. Strategy pattern được dùng cho các hành vi thay đổi theo từng loại thiết bị: họ lớp Sensor định nghĩa chiến lược đọc giá trị qua hàm getIndicator(), còn họ lớp TransEquipment định nghĩa chiến lược truyền dữ liệu qua hàm send(String data). Nhờ đó, mỗi loại cảm biến và mỗi loại thiết bị truyền tin có thể cài đặt thuật toán riêng mà không làm thay đổi lớp Instrument.

Template Method pattern được dùng trong lớp Instrument. Phương thức loop() là thuật toán khung cố định, lần lượt thực hiện các bước: getTime() -> readSensor() -> saveData() -> sendData(). Phần thay đổi giữa các loại dụng cụ nằm ở readSensor(), được cài đặt khác nhau trong Handy và Station. Thiết kế này bám sát yêu cầu đề bài, đồng thời giúp hệ thống dễ mở rộng, dễ bảo trì và giảm phụ thuộc giữa các thành phần.

3.1. Class diagram

Figure 1. Class diagram for InstrumentProject based on Strategy Pattern and Template Method Pattern

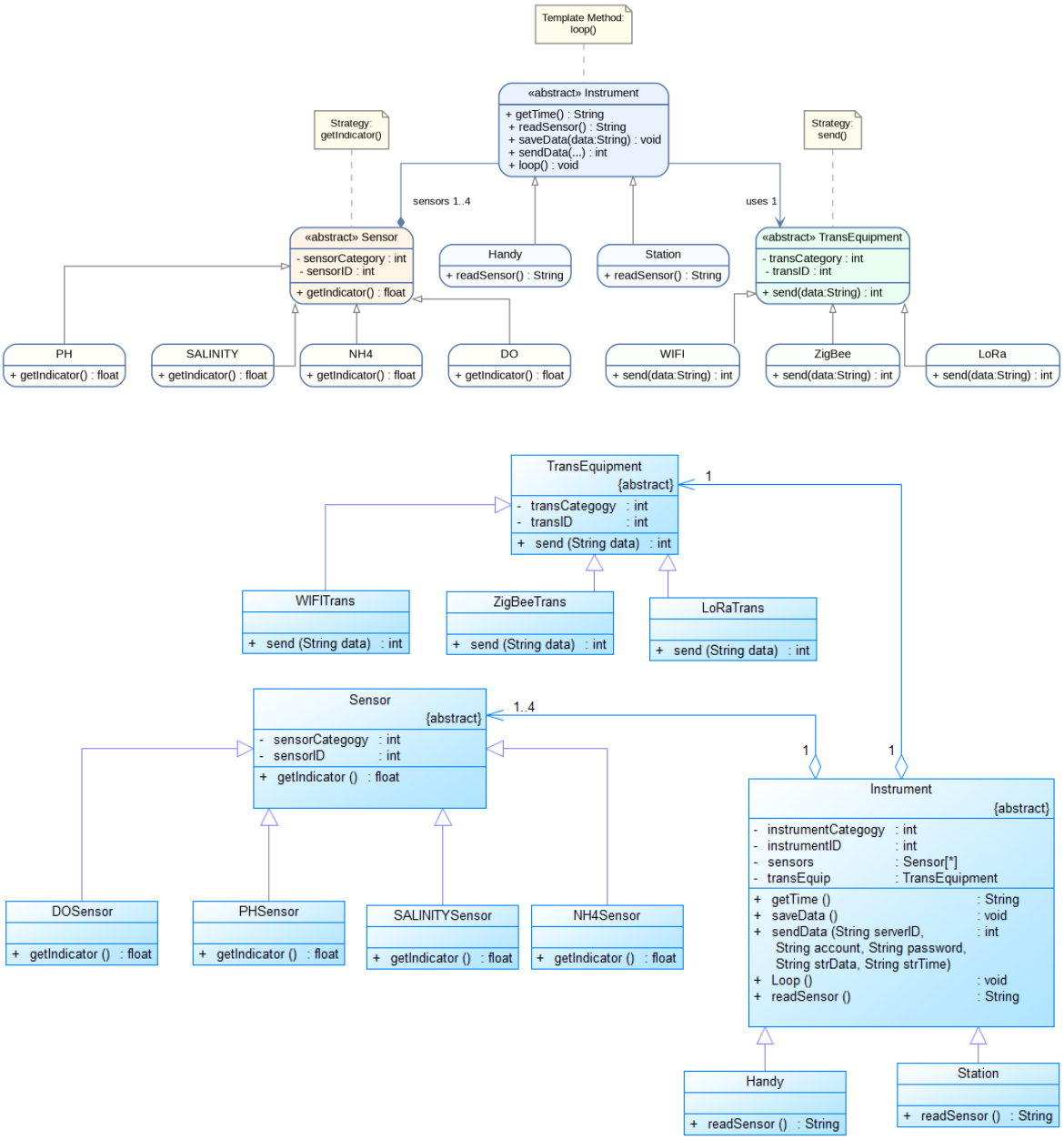

Figure 1. Class diagram for InstrumentProject

3.2. Class Descriptions

Theo ghi chú của đề, mục này chỉ cần mô tả lớp Instrument và các lớp con trực tiếp của nó là Handy và Station.

3.2.1. Class: Instrument

Purpose: Quản lý các loại dụng cụ đo môi trường và định nghĩa khung xử lý chung cho việc đọc, lưu và truyền dữ liệu.

Constraints: Lớp trừu tượng; không khởi tạo trực tiếp. Dụng cụ phải có ít nhất một cảm biến và một thiết bị truyền tin.

Persistent: No

Attribute Descriptions

1. Attribute: instrumentCategory
Type: int
Description: Định danh loại dụng cụ.
Constraints: Không.

2. Attribute: instrumentID
Type: int
Description: Định danh dụng cụ.
Constraints: Không.

3. Attribute: sensors
Type: ArrayList<Sensor>
Description: Danh sách cảm biến gắn vào dụng cụ.
Constraints: Handy chỉ có 1 cảm biến; Station có từ 1 đến 4 cảm biến.

4. Attribute: transEquipment
Type: TransEquipment
Description: Thiết bị truyền tin gắn với dụng cụ.
Constraints: Phải khác null trước khi gọi sendData().

Method Descriptions

1. Method: getTime()
Return Type: String
Parameters: Không
Return value: Chuỗi thời gian theo định dạng YYYY-MM-DD-hh:mm-ss
Pre-condition: Không
Post-condition: Trả về thời gian hiện tại
Attributes read/used: Không
Methods called: Không
Processing logic: Đọc thời gian thực của hệ thống và chuyển thành chuỗi.
Test case: Gọi hàm và kiểm tra chuỗi thời gian đúng định dạng.

2. Method: readSensor()
Return Type: String
Parameters: Không
Return value: Chuỗi JSON chứa giá trị đọc từ cảm biến
Pre-condition: Dụng cụ đã được gắn cảm biến
Post-condition: Trả về dữ liệu JSON
Attributes read/used: sensors
Methods called: getIndicator()
Processing logic: Đây là phương thức trừu tượng. Lớp con sẽ cài đặt cách đọc dữ liệu từ danh sách cảm biến và ghép thành chuỗi JSON.
Test case: Gắn cảm biến hợp lệ và kiểm tra chuỗi JSON trả về.

3. Method: saveData(data)
Return Type: void
Parameters: data : String
Return value: Không
Pre-condition: data khác rỗng
Post-condition: Dữ liệu được lưu vào bộ nhớ/thẻ nhớ
Attributes read/used: Không

Methods called: Không
Processing logic: Nhận chuỗi dữ liệu JSON và lưu vào bộ nhớ cục bộ.
Test case: Gọi với chuỗi JSON mẫu và kiểm tra thao tác lưu.

4. Method: sendData(...)
Return Type: int
Parameters: serverID, account, password, strData, strTime
Return value: 0 nếu thành công, khác 0 nếu lỗi
Pre-condition: transEquipment đã được gắn; dữ liệu truyền hợp lệ
Post-condition: Dữ liệu được chuyển cho thiết bị truyền tin để gửi về server
Attributes read/used: transEquipment
Methods called: send(String data)
Processing logic: Đóng gói dữ liệu cần truyền và gọi send() của thiết bị truyền tin tương ứng.
Test case: Gắn WIFI, ZigBee hoặc LoRa và kiểm tra mã trả về.

5. Method: loop()
Return Type: void
Parameters: Không
Return value: Không
Pre-condition: Dụng cụ đã gắn cảm biến và thiết bị truyền tin
Post-condition: Hoàn thành chu trình đọc - lưu - gửi dữ liệu
Attributes read/used: sensors, transEquipment
Methods called: getTime(), readSensor(), saveData(), sendData()
Processing logic: Đây là template method, cố định trình tự xử lý của hệ thống: đọc thời gian, đọc cảm biến, lưu dữ liệu, truyền dữ liệu.
Test case: Gọi loop() trên Handy hoặc Station và quan sát toàn bộ luồng xử lý.

3.2.2. Class: Handy

Purpose: Quản lý dụng cụ cầm tay.

Constraints: Chỉ gắn đúng 1 cảm biến.

Persistent: No

Method Descriptions

1. Method: readSensor()
Return Type: String
Parameters: Không
Return value: Chuỗi JSON chứa giá trị của duy nhất một cảm biến
Pre-condition: Danh sách sensors có đúng 1 phần tử
Post-condition: Trả về JSON chứa 1 giá trị cảm biến
Attributes read/used: sensors
Methods called: getIndicator()
Processing logic: Gọi getIndicator() của cảm biến duy nhất được gắn vào Handy, sau đó đóng gói kết quả thành JSON.
Test case: Gắn một cảm biến PH hoặc DO và kiểm tra JSON trả về chỉ có một trường dữ liệu.

3.2.3. Class: Station

Purpose: Quản lý trạm đo cố định.

Constraints: Gắn từ 1 đến 4 cảm biến.

Persistent: No

Method Descriptions

1. Method: readSensor()
Return Type: String
Parameters: Không
Return value: Chuỗi JSON chứa giá trị của các cảm biến đang gắn trên Station
Pre-condition: Danh sách sensors có từ 1 đến 4 phần tử
Post-condition: Trả về JSON chứa nhiều giá trị cảm biến
Attributes read/used: sensors
Methods called: getIndicator()
Processing logic: Lần lượt duyệt qua danh sách cảm biến, gọi getIndicator() cho từng cảm biến rồi ghép các kết quả thành chuỗi JSON tổng hợp.
Test case: Gắn nhiều cảm biến như DO, PH, NH4 và kiểm tra JSON trả về đủ các giá trị.

4. Consequence

• Thiết kế giúp hệ thống dễ mở rộng. Khi cần thêm loại cảm biến mới hoặc loại thiết bị truyền tin mới, chỉ cần tạo thêm lớp con của Sensor hoặc TransEquipment mà không phải sửa nhiều ở Instrument.

• Thiết kế giúp hệ thống dễ bảo trì vì phần khung xử lý chung được gom vào Instrument.loop(), còn phần khác biệt được tách ra thành các chiến lược riêng. Handy và Station chỉ tập trung vào cách đọc cảm biến; WIFI, ZigBee và LoRa chỉ tập trung vào cách gửi dữ liệu.

• Thiết kế làm giảm phụ thuộc giữa các thành phần. Instrument không cần biết chi tiết thuật toán đo của từng loại sensor hay cơ chế gửi của từng loại transEquipment. Đổi lại, số lượng lớp sẽ nhiều hơn so với cách cài đặt trực tiếp, nhưng đây là đánh đổi phù hợp để tăng tính mở rộng và tái sử dụng.

5. Implementation (Java code example)

Code minh họa dưới đây bám sát sơ đồ lớp và hai design pattern đã chọn. Khi nộp bài có thể tách từng lớp thành các file .java riêng trong project Instrument.

```
package instrument;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;

abstract class Sensor {
    protected int sensorCategory;
    protected int sensorID;

    public Sensor(int sensorCategory, int sensorID) {
        this.sensorCategory = sensorCategory;
        this.sensorID = sensorID;
    }

    public abstract float getIndicator();
}

class DO extends Sensor {
    public DO(int sensorCategory, int sensorID) { super(sensorCategory, sensorID); }
    @Override public float getIndicator() { return 5.8f; }
}

class PH extends Sensor {
    public PH(int sensorCategory, int sensorID) { super(sensorCategory, sensorID); }
    @Override public float getIndicator() { return 7.2f; }
}
```

```java
class SALINITY extends Sensor {
    public SALINITY(int sensorCategory, int sensorID) { super(sensorCategory, sensorID); }
    @Override public float getIndicator() { return 12.5f; }
}

class NH4 extends Sensor {
    public NH4(int sensorCategory, int sensorID) { super(sensorCategory, sensorID); }
    @Override public float getIndicator() { return 0.9f; }
}

abstract class TransEquipment {
    protected int transCategory;
    protected int transID;

    public TransEquipment(int transCategory, int transID) {
        this.transCategory = transCategory;
        this.transID = transID;
    }

    public abstract int send(String data);
}

class WIFI extends TransEquipment {
    public WIFI(int transCategory, int transID) { super(transCategory, transID); }
    @Override public int send(String data) {
        System.out.println("WIFI sending: " + data);
        return 0;
    }
}

class ZigBee extends TransEquipment {
    public ZigBee(int transCategory, int transID) { super(transCategory, transID); }
    @Override public int send(String data) {
        System.out.println("ZigBee sending: " + data);
        return 0;
    }
}

class LoRa extends TransEquipment {
    public LoRa(int transCategory, int transID) { super(transCategory, transID); }
    @Override public int send(String data) {
        System.out.println("LoRa sending: " + data);
        return 0;
    }
}

abstract class Instrument {
    protected int instrumentCategory;
    protected int instrumentID;
    protected ArrayList<Sensor> sensors;
    protected TransEquipment transEquipment;

    public Instrument(int instrumentCategory, int instrumentID,
                      ArrayList<Sensor> sensors, TransEquipment transEquipment) {
        this.instrumentCategory = instrumentCategory;
        this.instrumentID = instrumentID;
        this.sensors = sensors;
        this.transEquipment = transEquipment;
    }

    public String getTime() {
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy-MM-dd-HH:mm-ss");
        return LocalDateTime.now().format(fmt);
    }

    public abstract String readSensor();

    public void saveData(String data) {
        System.out.println("Saving data to memory card: " + data);
    }
```

```
    public int sendData(String serverID, String account, String password,
                        String strData, String strTime) {
        String payload = "{\"serverID\":\"" + serverID + "\","
                + "\"account\":\"" + account + "\","
                + "\"time\":\"" + strTime + "\","
                + "\"data\":" + strData + "}";
        return transEquipment.send(payload);
    }

    public final void loop() {
        String time = getTime();
        String data = readSensor();
        saveData(data);
        int status = sendData("DataServer01", "iotuser", "123456", data, time);
        System.out.println("Send status = " + status);
    }
}

class Handy extends Instrument {
    public Handy(int instrumentCategory, int instrumentID,
                 ArrayList<Sensor> sensors, TransEquipment transEquipment) {
        super(instrumentCategory, instrumentID, sensors, transEquipment);
    }

    @Override
    public String readSensor() {
        if (sensors == null || sensors.size() != 1) {
            throw new IllegalArgumentException("Handy must have exactly 1 sensor.");
        }
        Sensor s = sensors.get(0);
        return "{\"instrumentID\":" + instrumentID + ","
                + "\"type\":\"Handy\","
                + "\"values\":{\"" + s.getClass().getSimpleName() + "\":" + s.getIndicator() + "}}";
    }
}

class Station extends Instrument {
    public Station(int instrumentCategory, int instrumentID,
                   ArrayList<Sensor> sensors, TransEquipment transEquipment) {
        super(instrumentCategory, instrumentID, sensors, transEquipment);
    }

    @Override
    public String readSensor() {
        if (sensors == null || sensors.size() < 1 || sensors.size() > 4) {
            throw new IllegalArgumentException("Station must have from 1 to 4 sensors.");
        }
        StringBuilder json = new StringBuilder();
        json.append("{\"instrumentID\":").append(instrumentID).append(",");
        json.append("\"type\":\"Station\",");
        json.append("\"values\":{");
        for (int i = 0; i < sensors.size(); i++) {
            Sensor s = sensors.get(i);
            json.append("\"").append(s.getClass().getSimpleName()).append("\":")
                .append(s.getIndicator());
            if (i < sensors.size() - 1) json.append(",");
        }
        json.append("}}");
        return json.toString();
    }
}
```

```java
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;

// ============================================================
// 1. FILE: Sensor.java (Strategy Pattern - Abstract)
// ============================================================
public abstract class Sensor {
    // Đề yêu cầu thuộc tính dạng tĩnh (static) [cite: 489-491]
    protected static int sensorCategory;
    protected static int sensorID;

    public abstract float getIndicator(); // [cite: 493]
}

// ============================================================
// 2. CÁC FILE LỚP CON CỦA SENSOR
// ============================================================
// File: DOSensor.java
class DOSensor extends Sensor {
    @Override
    public float getIndicator() { return 5.8f; }
}

// File: PHSensor.java
class PHSensor extends Sensor {
    @Override
    public float getIndicator() { return 7.2f; }
}

// File: SALINITYSensor.java
class SALINITYSensor extends Sensor {
    @Override
    public float getIndicator() { return 12.5f; }
}

// File: NH4Sensor.java
class NH4Sensor extends Sensor {
    @Override
    public float getIndicator() { return 0.9f; }
}

// ============================================================
// 3. FILE: TransEquipment.java (Strategy Pattern - Abstract)
// ============================================================
public abstract class TransEquipment {
    // Đề yêu cầu thuộc tính dạng tĩnh (static) [cite: 496-498]
    protected static int transCategory;
    protected static int transID;

    public abstract int send(String data); // [cite: 500]
}

// ============================================================
// 4. CÁC FILE LỚP CON CỦA TRANSEQUIPMENT
// ============================================================
// File: WIFITrans.java
class WIFITrans extends TransEquipment {
    @Override
    public int send(String data) { return 0; } // 0: truyền thành công [cite: 500]
}

// File: ZigBeeTrans.java
class ZigBeeTrans extends TransEquipment {
    @Override
    public int send(String data) { return 0; }
}

// File: LoRaTrans.java
```

```java
class LoRaTrans extends TransEquipment {
    @Override
    public int send(String data) { return 0; }
}

// ============================================================
// 5. FILE: Instrument.java (Template Method Pattern - Abstract)
// ============================================================
public abstract class Instrument {
    protected int instrumentCategory;
    protected int instrumentID;
    protected ArrayList<Sensor> sensors;
    protected TransEquipment transEquip;

    public Instrument(int instrumentCategory, int instrumentID, TransEquipment transEquip) {
        this.instrumentCategory = instrumentCategory;
        this.instrumentID = instrumentID;
        this.sensors = new ArrayList<>(); // Khởi tạo danh sách cảm biến
        this.transEquip = transEquip;
    }

    // Hàm hỗ trợ gắn thêm cảm biến vào dụng cụ
    public void attachSensor(Sensor s) {
        this.sensors.add(s);
    }

    public String getTime() { // [cite: 510]
        DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy-MM-dd-HH:mm-ss");
        return LocalDateTime.now().format(fmt);
    }

    public abstract String readSensor(); // Trừu tượng vì Handy và Station đọc khác nhau [cite: 511-512]

    public void saveData(String data) { // [cite: 514]
        System.out.println("Saving data to memory: " + data);
    }

    public int sendData(String serverID, String account, String password, String strData, String strTime)
{ // [cite: 516-517]
        return transEquip.send(strData); // Gọi hàm send của thiết bị truyền kết nối [cite: 528]
    }

    // Template Method định nghĩa thuật toán khung cố định [cite: 529]
    public final void Loop() {
        String time = getTime();
        String data = readSensor();
        saveData(data);
        int status = sendData("Server01", "admin", "123456", data, time);
        System.out.println("Truyền dữ liệu xong. Trạng thái: " + status);
    }
}

// ============================================================
// 6. FILE: Handy.java (Dụng cụ cầm tay)
// ============================================================
class Handy extends Instrument {
    public Handy(int instrumentCategory, int instrumentID, TransEquipment transEquip) {
        super(instrumentCategory, instrumentID, transEquip);
    }

    @Override
    public String readSensor() {
        if (sensors.isEmpty()) return "{}";
        // Handy chỉ gắn được 1 cảm biến[cite: 502], nên lấy phần tử đầu tiên
        Sensor s = sensors.get(0);
        return "{\"HandyData\": " + s.getIndicator() + "}"; // Trả về dạng JSON [cite: 512]
    }
}

// ============================================================
```

```java
// 7. FILE: Station.java (Trạm đo cố định)
// ==========================================================
class Station extends Instrument {
    public Station(int instrumentCategory, int instrumentID, TransEquipment transEquip) {
        super(instrumentCategory, instrumentID, transEquip);
    }

    @Override
    public String readSensor() {
        // Station gắn được 1 đến 4 cảm biến[cite: 503], phải dùng vòng lặp để đọc hết
        StringBuilder json = new StringBuilder("{");
        for (int i = 0; i < sensors.size(); i++) {
            json.append("\"Sensor_").append(i+1).append("\": ")
                .append(sensors.get(i).getIndicator());
            if (i < sensors.size() - 1) json.append(", ");
        }
        json.append("}");
        return json.toString(); // Trả về chuỗi định dạng JSON [cite: 512]
    }
}
```